ESSENTIALS OF DATA SCIENCE

Gaurav Kotkar | Roll No: 202501100051 (s13)

Practical No. 1: Python Fundamentals

1. Momentum Calculation

Accepting mass and velocity to calculate momentum based on the assigned formula $e = mc^2$.

```
mass = float(input('Enter object mass (kg): '))
vel = float(input('Enter velocity (m/s): '))
# Applying syllabus formula
result_momentum = mass * (vel ** 2)
print(f'Calculated Momentum: {result_momentum}')
```

2. Numeric Operations based on Digits

```
import math
val = int(input('Enter an integer: '))
if 0 <= val <= 9:
    print(f'Square: {val**2}')
elif 10 <= val <= 99:
    print(f'Square Root: {math.sqrt(val):.2f}')
elif 100 <= val <= 999:
    print(f'Cube Root: {val**(1/3):.2f}')
else:
    print('Input is outside the 1-3 digit range.')
```

3. Data Transformation (Age & Currency)

```
from datetime import date

def process_info(dob, sal_inr):
    curr = date.today()
    age = curr.year - dob.year - ((curr.month, curr.day) < (dob.month, dob.day))
    sal_usd = sal_inr / 83.0
    return age, sal_usd
```

```
birth = date(1998, 8, 15)
inr_val = 650000
final_age, final_usd = process_info(birth, inr_val)
print(f'Age: {final_age}, Salary: ${final_usd:.2f}')
```

4. Number Reversal

```
n = int(input('Enter number to reverse: '))
rev_n = int(str(n)[::-1])
print(f'Result: {rev_n}')
```

Academic Grading System

Computing results for 5 courses with specific grading criteria.

```
scores = [float(input(f'Subject {i+1} Marks: ')) for i in range(5)]

if any(s < 40 for s in scores):
    print('Result: Fail (Below Minimum Criteria)')
else:
    percentage = sum(scores) / 5
    if percentage > 75: rank = 'Distinction'
    elif percentage >= 60: rank = 'First Division'
    elif percentage >= 50: rank = 'Second Division'
    elif percentage >= 40: rank = 'Third Division'
    print(f'Total Percentage: {percentage}% | Grade: {rank}')
```

Practical No. 2: Data Structures

Fibonacci Sequence (Recursive)

```
def get_fib(n):
    if n <= 1: return n
    return get_fib(n-1) + get_fib(n-2)

limit = 10
sequence = [get_fib(i) for i in range(limit)]
print(f'Fibonacci Sequence: {sequence}')
```

Pattern Generation

```
n_rows = 5
for i in range(1, n_rows + 1):
    print('*' * i)
```

Practical No. 3: NumPy & Matrix Operations

```
import numpy as np

# Matrix initialization
matrix_a = np.array([[15, 25], [35, 45]])

print("Transpose Matrix:")
print(matrix_a.T)

print("
Vertical Concatenation:")
print(np.vstack((matrix_a, matrix_a)))

print(f"
Sum across columns: {np.sum(matrix_a, axis=0)}")
```

Practical No. 4: Pandas Analysis

Sales Data Analysis

```
import pandas as pd

raw_data = {
      'Month': ['Jan', 'Feb', 'Mar', 'Jan', 'Feb'],
      'Sales': [500, 200, 300, 450, 150],
      'City': ['Mumbai', 'Pune', 'Mumbai', 'Pune', 'Mumbai']
}
sales_df = pd.DataFrame(raw_data)

top_month = sales_df.groupby('Month')['Sales'].sum().idxmax()
print(f'Highest Sales Month: {top_month}')
```

Practical No. 5: Visualization

Titanic Dataset Visuals

```
import matplotlib.pyplot as plt
import seaborn as sns

df_titanic = sns.load_dataset('titanic')

# Visualization: Survival Count
df_titanic['survived'].value_counts().plot(kind='bar', color=['red', 'green'])
plt.title('Passenger Survival Distribution')
plt.xlabel('Survived (0=No, 1=Yes)')
plt.ylabel('Count')
plt.show()
```